

Q1. What is the maximum number of matches played by an individual player in a season? Print the player name along with the number of matched played.

```
In [1]: import pandas as pd

file_path = r"C:\Users\pc\OneDrive\Desktop\Assignments\EDA\Worksheet in C  Users pc Downloads Assignment(1) (1) (1).x
excel_file = pd.ExcelFile(file_path)

sheet_names = excel_file.sheet_names
sheet_names
```

Out[1]: ['IPL_Dataset']

```
In [2]: df_ipl = excel_file.parse('IPL_Dataset')

df_ipl.head()
```

Out[2]:

	Jersey No	Player	Matches	Inns	Not Out	Runs	Highest Score	Avg	Balls faced	Strike rate	100	50	4s	6s
0	1	KL Rahul	14	14	2	670	132*	55.83	518	129.34	1	5	58	23
1	2	Shikhar Dhawan	17	17	3	618	106*	44.14	427	144.73	2	4	67	12
2	3	David Warner	16	16	2	548	85*	39.14	407	134.64	0	4	52	14
3	4	Shreyas Iyer	17	17	2	519	88*	34.60	421	123.27	0	3	40	16
4	5	Ishan Kishan	14	13	4	516	99	57.33	354	145.76	0	4	36	30

```
In [3]: max_matches = df_ipl['Matches'].max()
players_with_max_matches = df_ipl[df_ipl['Matches'] == max_matches][['Player', 'Matches']]

players_with_max_matches
```

Out[3]:

	Player	Matches
1	Shikhar Dhawan	17
3	Shreyas Iyer	17
19	Marcus Stoinis	17
67	Kagiso Rabada	17

Q2. Top 2 players with maximum Average who have scored atleast 2 half centuries ?

```
In [4]: df_ipl.columns
```

Out[4]: Index(['Jersey No', 'Player', 'Matches', 'Inns', 'Not Out', 'Runs', 'Highest Score', 'Avg', 'Balls faced', 'Strike rate', '100', '50', '4s', '6s'], dtype='object')

```
In [5]: players_with_2_half_centuries = df_ipl[df_ipl[50] >= 2]

top_avg_players = players_with_2_half_centuries.sort_values(by='Avg', ascending=False).head(2)

top_avg_players[['Player', 'Avg', 50]]
```

Out[5]:

	Player	Avg	50
36	Wriddhiman Saha	71.33	2
4	Ishan Kishan	57.33	4

Q3. Create 2 new columns based on Player name. First column will have first name and second column will have last name. Eg: for the player Shikhar Dhawan, Shikhar will be the first name and Dhawan will be the last name.

```
In [6]: df_ipl['First Name'] = df_ipl['Player'].apply(lambda x: x.split()[0])
df_ipl['Last Name'] = df_ipl['Player'].apply(lambda x: ' '.join(x.split()[1:]) if len(x.split()) > 1 else '')

df_ipl[['Player', 'First Name', 'Last Name']].head()
```

Out[6]:

	Player	First Name	Last Name
0	KL Rahul	KL	Rahul
1	Shikhar Dhawan	Shikhar	Dhawan
2	David Warner	David	Warner
3	Shreyas Iyer	Shreyas	Iyer
4	Ishan Kishan	Ishan	Kishan

Q4. Create a new column (Cleaned_Highest_score) based on Highest score variable. Remove the Asterik(*) mark and convert the data type into INT.

In [7]:

```
df_ip1['Cleaned_Highest_score'] = df_ip1['Highest Score'].astype(str).str.replace('*', '', regex=False).astype(int)
df_ip1[['Highest Score', 'Cleaned_Highest_score']].head()
```

Out[7]:

	Highest Score	Cleaned_Highest_score
0	132*	132
1	106*	106
2	85*	85
3	88*	88
4	99	99

Q5. Print the total number of centuries scored in the entire season.

In [8]:

```
total_centuries = df_ip1[100].sum()
total_centuries
```

Out[8]: 5

Q6. Print all the player names whose strike rate is less than the average strike rate of all players in entire season. Print the player name, his strike rate and average strike rate.

In [9]:

```
average_strike_rate = df_ip1['Strike rate'].mean()

below_avg_strike_rate = df_ip1[df_ip1['Strike rate'] < average_strike_rate][['Player', 'Strike rate']]

below_avg_strike_rate['Average Strike Rate'] = average_strike_rate

below_avg_strike_rate
```

Out[9]:

	Player	Strike rate	Average Strike Rate
51	Ajinkya Rahane	105.60	107.364737
55	Glenn Maxwell	101.88	107.364737
58	Vijay Shankar	101.04	107.364737
61	Josh Philippe	101.29	107.364737
62	Gurkeerat Singh	88.75	107.364737
65	Kedar Jadhav	93.93	107.364737
70	Yashasvi Jaiswal	90.90	107.364737
71	Shreyas Gopal	94.87	107.364737
77	Murali Vijay	74.41	107.364737
79	Chris Jordan	93.54	107.364737
80	Navdeep Saini	100.00	107.364737
82	Kamlesh Nagarkoti	70.96	107.364737
84	Harshal Patel	87.50	107.364737
85	Jimmy Neesham	105.55	107.364737
86	Tom Banton	90.00	107.364737
89	Prabhsimran Singh	100.00	107.364737
92	Kuldeep Yadav	61.90	107.364737
94	Moeen Ali	75.00	107.364737
95	Sandeep Sharma	80.00	107.364737
96	Shardul Thakur	57.14	107.364737
98	Rinku Singh	100.00	107.364737
99	Shivam Mavi	71.42	107.364737
100	Varun Chakaravorthy	66.66	107.364737
101	Jaydev Unadkat	69.23	107.364737
102	Ankit Rajpoot	90.00	107.364737
104	Shahbaz Nadeem	87.50	107.364737
105	Pravin Dubey	53.84	107.364737
107	Deepak Chahar	58.33	107.364737
108	Ravi Bishnoi	58.33	107.364737
109	Andrew Tye	100.00	107.364737
111	Kartik Tyagi	66.66	107.364737
112	Murugan Ashwin	100.00	107.364737
114	T Natarajan	60.00	107.364737
115	Prasidh Krishna	50.00	107.364737
116	Rahul Chahar	50.00	107.364737
117	Mohammad Shami	66.66	107.364737
118	Nikhil Naik	33.33	107.364737
119	Mujeeb Ur Rahman	33.33	107.364737
120	Dale Steyn	50.00	107.364737
121	Varun Aaron	10.00	107.364737
122	Shahbaz Ahmed	100.00	107.364737
123	Yuzvendra Chahal	33.33	107.364737
124	Mitchell Marsh	0.00	107.364737
125	Umesh Yadav	0.00	107.364737
126	Bhuvneshwar Kumar	0.00	107.364737
127	Sheldon Cottrell	0.00	107.364737
128	Khaleel Ahmed	0.00	107.364737
129	Arshdeep Singh	0.00	107.364737
130	Daniel Sams	0.00	107.364737
131	Shreevats Goswami	0.00	107.364737

	Player	Strike rate	Average Strike Rate
132	Trent Boult	0.00	107.364737

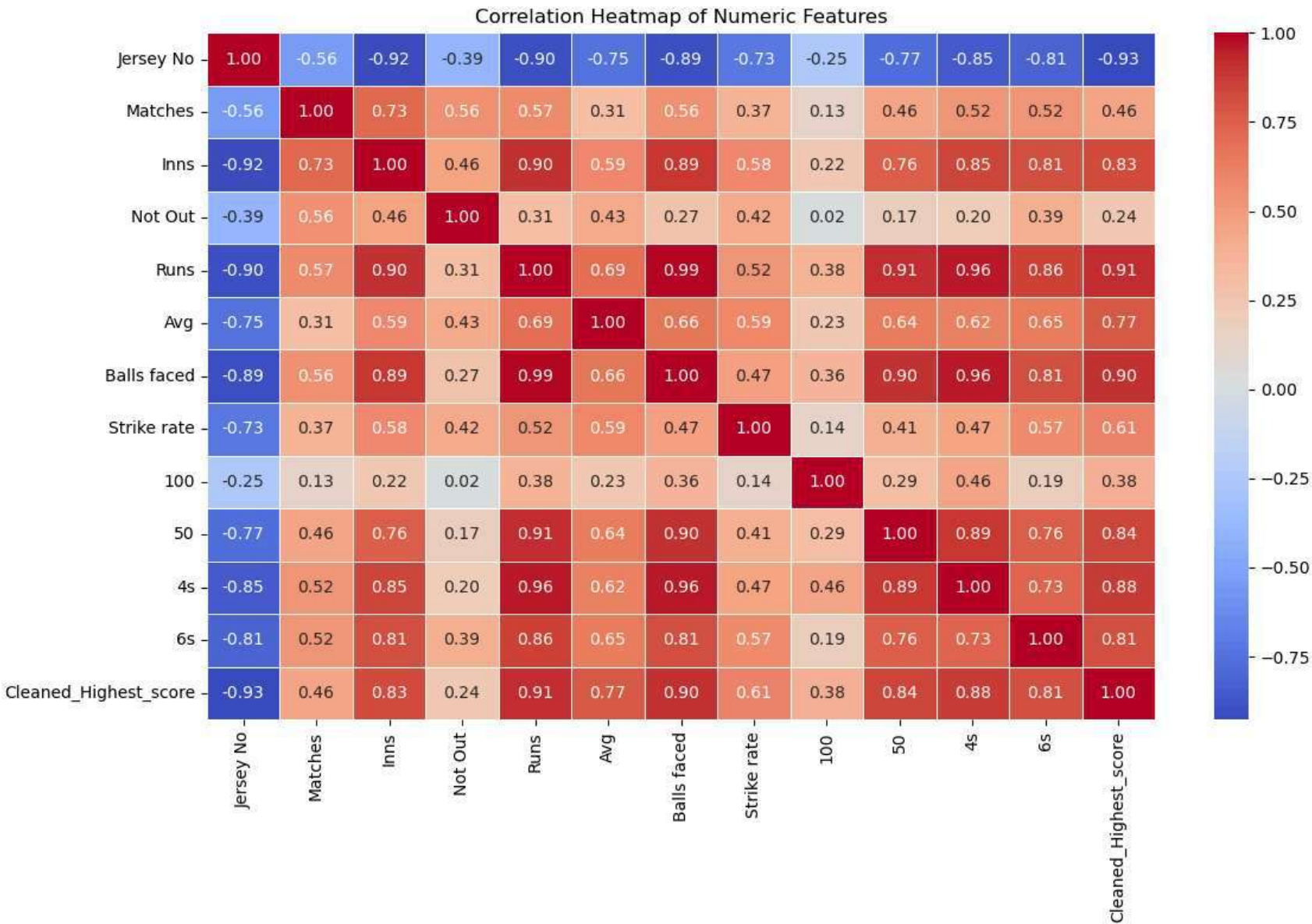
Q7. Please check the correlation between the features and create a heat map.

```
In [10]: import seaborn as sns
import matplotlib.pyplot as plt

numeric_df = df_ipl.select_dtypes(include='number')

correlation_matrix = numeric_df.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap of Numeric Features")
plt.tight_layout()
plt.show()
```



Q8. Check the list of players who has an average greater than 50 as well strike rate above 120. Print player name, average and strike rate.

```
In [11]: high_avg_strike_players = df_ipl[(df_ipl['Avg'] > 50) & (df_ipl['Strike rate'] > 120)]

high_avg_strike_players[['Player', 'Avg', 'Strike rate']]
```

Out[11]:

	Player	Avg	Strike rate
0	KL Rahul	55.83	129.34
4	Ishan Kishan	57.33	145.76
31	Kieron Pollard	53.60	191.42
36	Wriddhiman Saha	71.33	139.86
37	Ruturaj Gaikwad	51.00	120.71
57	Deepak Hooda	101.00	142.25
60	Tom Curran	83.00	133.87

Q9. Please check the list of players who has an average greater than 40 and balls faced above 100. Print player name, average and balls faced.

```
In [12]: high_avg_balls_players = df_ipl[(df_ipl['Avg'] > 40) & (df_ipl['Balls faced'] > 100)]

high_avg_balls_players[['Player', 'Avg', 'Balls faced']]
```


Out[12]:

	Player	Avg	Balls faced
0	KL Rahul	55.83	518
1	Shikhar Dhawan	44.14	427
4	Ishan Kishan	57.33	354
8	Virat Kohli	42.36	384
9	ABD Villiers	45.40	286
10	Faf Duplessis	40.81	319
14	Eoin Morgan	41.80	302
24	Kane Williamson	45.28	237
27	Chris Gayle	41.14	210
28	Ben Stokes	40.71	200
31	Kieron Pollard	53.60	140
32	Rahul Tewatia	42.50	183
33	Ravindra Jadeja	46.40	135
36	Wriddhiman Saha	71.33	153
37	Ruturaj Gaikwad	51.00	169

Q10. Players who scored atleast one century in this season. Create visualization.

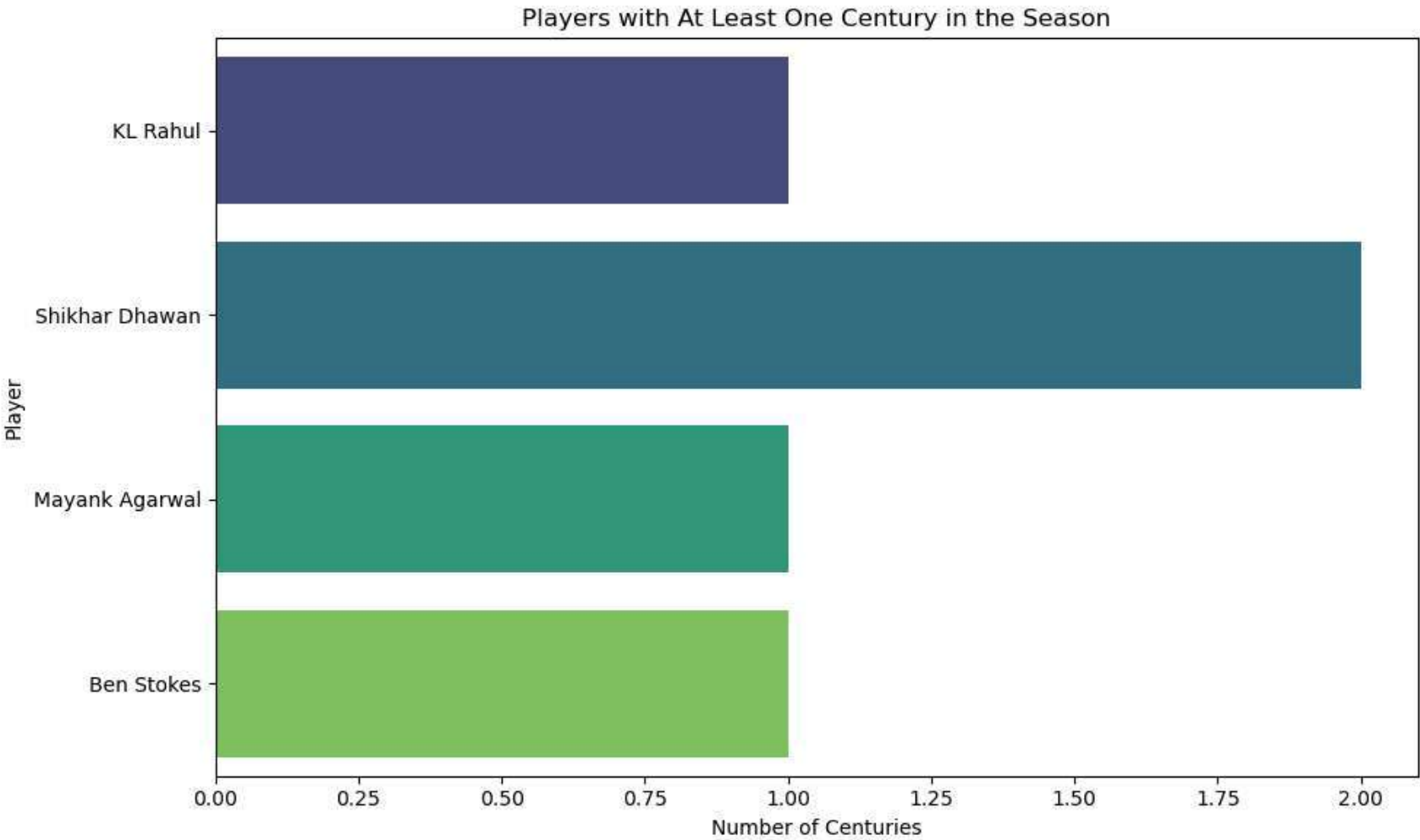
```
In [13]: century_players = df_ipl[df_ipl[100] >= 1][['Player', 100]]

plt.figure(figsize=(10, 6))
sns.barplot(data=century_players, x=100, y='Player', palette='viridis')
plt.xlabel("Number of Centuries")
plt.ylabel("Player")
plt.title("Players with At Least One Century in the Season")
plt.tight_layout()
plt.show()
```

C:\Users\pc\AppData\Local\Temp\ipykernel_14496\4044861563.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=century_players, x=100, y='Player', palette='viridis')
```



Q11. Players who scored atleast 4 half centuries in this season.

```
In [14]: half_century_players = df_ipl[df_ipl[50] >= 4][['Player', 50]]

half_century_players
```

Out[14]:

	Player	50
0	KL Rahul	5
1	Shikhar Dhawan	4
2	David Warner	4
4	Ishan Kishan	4
5	Quinton Kock	4
6	Suryakumar Yadav	4
7	Devdutt Padikkal	5
9	ABD Villiers	5
10	Faf Duplessis	4

Q12. Check the list of players who hit more than 45 boundaries and more than 10 sixes in this season.

In [15]:

```
boundary_hitters = df_ip1[(df_ip1['4s'] > 45) & (df_ip1['6s'] > 10)][['Player', '4s', '6s']]

boundary_hitters
```

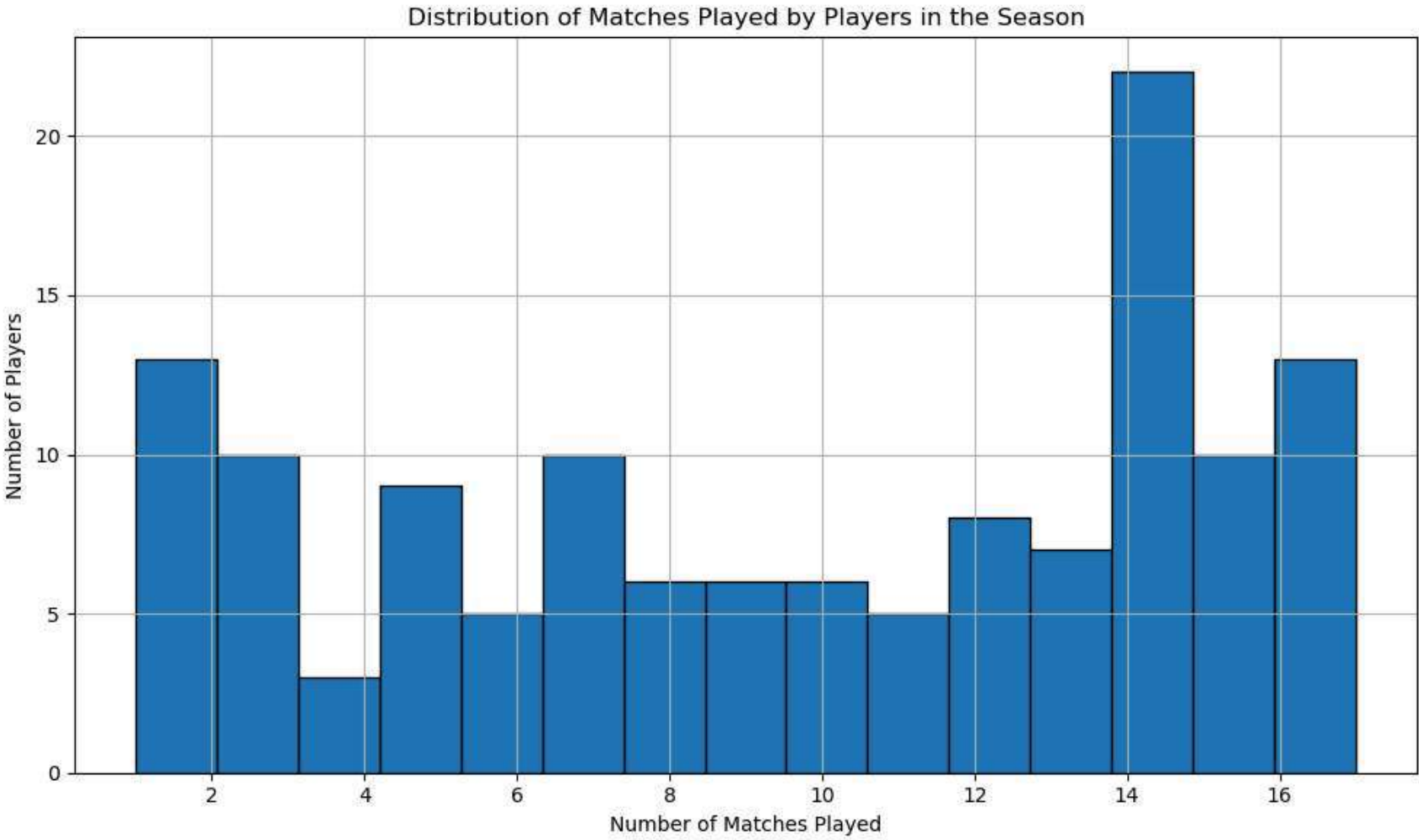
Out[15]:

	Player	4s	6s
0	KL Rahul	58	23
1	Shikhar Dhawan	67	12
2	David Warner	52	14
5	Quinton Kock	46	22
6	Suryakumar Yadav	61	11

Q13. Plot a histogram of number of matches played in a season by players.

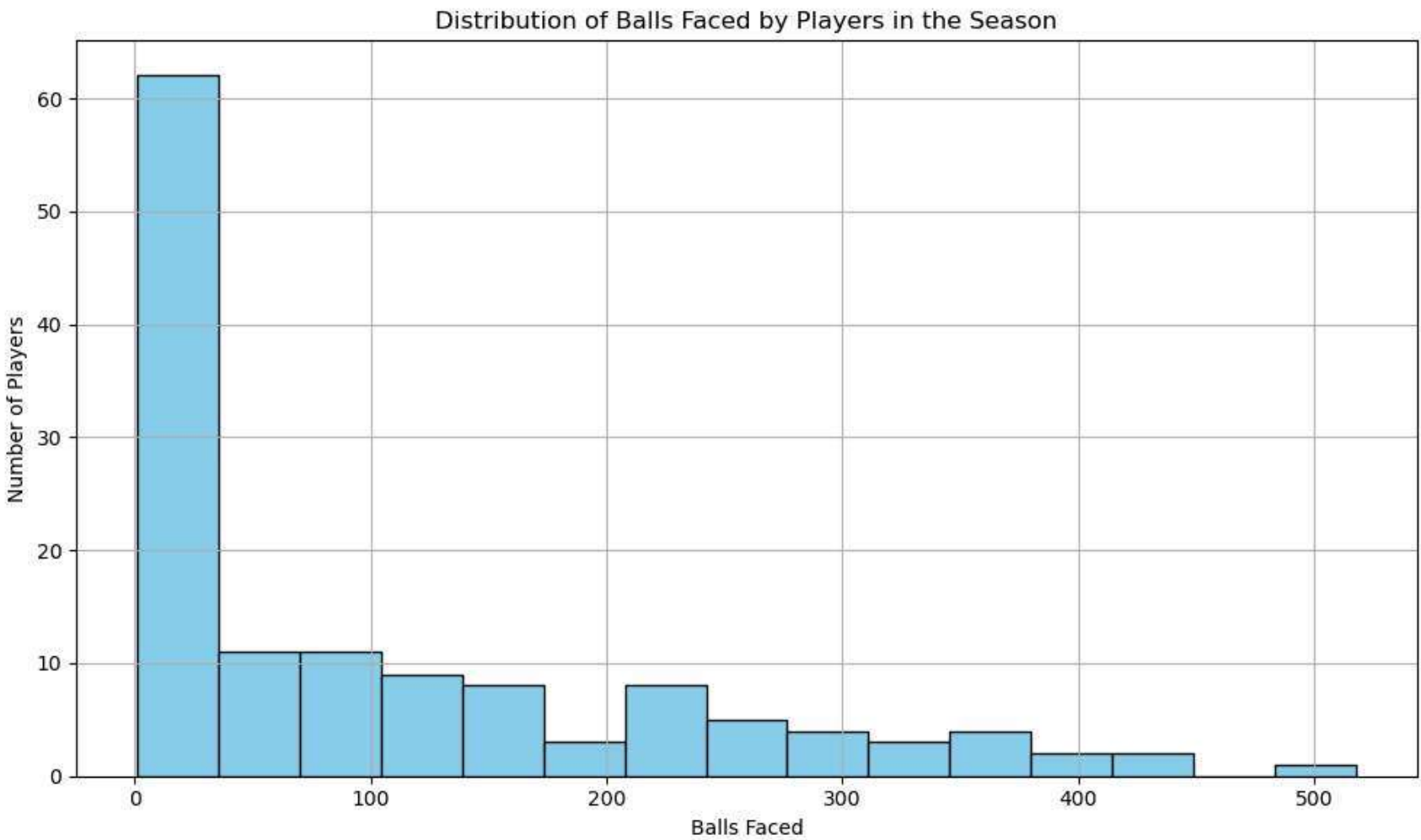
In [16]:

```
plt.figure(figsize=(10, 6))
plt.hist(df_ip1['Matches'], bins=15, edgecolor='black')
plt.xlabel('Number of Matches Played')
plt.ylabel('Number of Players')
plt.title('Distribution of Matches Played by Players in the Season')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Q14. Plot the histogram of balls faced by players.

```
In [17]: plt.figure(figsize=(10, 6))
plt.hist(df_ipl['Balls faced'], bins=15, edgecolor='black', color='skyblue')
plt.xlabel('Balls Faced')
plt.ylabel('Number of Players')
plt.title('Distribution of Balls Faced by Players in the Season')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Q15. Top 10 players with most runs in a season.

```
In [18]: top_run_scorers = df_ipl.sort_values(by='Runs', ascending=False).head(10)[['Player', 'Runs']]
top_run_scorers
```

Out[18]:

	Player	Runs
0	KL Rahul	670
1	Shikhar Dhawan	618
2	David Warner	548
3	Shreyas Iyer	519
4	Ishan Kishan	516
5	Quinton Kock	503
6	Suryakumar Yadav	480
7	Devdutt Padikkal	473
8	Virat Kohli	466
9	ABD Villiers	454

Q16. Print the players who played the match but didn't get the batting.

```
In [19]: non_batting_players = df_ipl[(df_ipl['Matches'] > 0) & (df_ipl['Inns'] == 0)][['Player', 'Matches', 'Inns']]
non_batting_players
```

Out[19]:

Player	Matches	Inns
--------	---------	------

There are no players in the dataset who played matches but did not get a chance to bat-i.e., every player who appeared in a match has at least one batting inning recorded.

Q17. Create a new column to show the percentage of total runs scored in 4s and 6s. Then print the top 5 players with maximum percentage.

```
In [20]: df_ipl['Runs from 4s and 6s'] = df_ipl['4s'] * 4 + df_ipl['6s'] * 6

df_ipl['Boundary Run %'] = (df_ipl['Runs from 4s and 6s'] / df_ipl['Runs']) * 100

top_boundary_percent = df_ipl.sort_values(by='Boundary Run %', ascending=False).head(5)[['Player', 'Runs', 'Runs from 4s and 6s', 'Boundary Run %']]
```

top_boundary_percent

Out[20]:

	Player	Runs	Runs from 4s and 6s	Boundary Run %
109	Andrew Tye	6	6	100.000000
48	Andre Russell	117	90	76.923077
74	Chris Morris	34	26	76.470588
29	Hardik Pandya	281	206	73.309609
47	Sunil Narine	121	88	72.727273

Q18. Print the players with top 5 Not out percentages (Not Out percentage can be calculated as number of Not outs divided by Innings).

In [21]:

```
df_ip1['Not Out %'] = (df_ip1['Not Out'] / df_ip1['Inns']) * 100

valid_not_outs = df_ip1[df_ip1['Inns'] > 0]

top_not_out_percent = valid_not_outs.sort_values(by='Not Out %', ascending=False).head(5)[['Player', 'Inns', 'Not Out']]

top_not_out_percent
```

Out[21]:

	Player	Inns	Not Out	Not Out %
122	Shahbaz Ahmed	1	1	100.0
97	Mohammad Nabi	1	1	100.0
114	T Natarajan	3	3	100.0
116	Rahul Chahar	1	1	100.0
113	Dhawal Kulkarni	1	1	100.0

Q19. Create visualization of top 10 players with highest number of sixes.

In [22]:

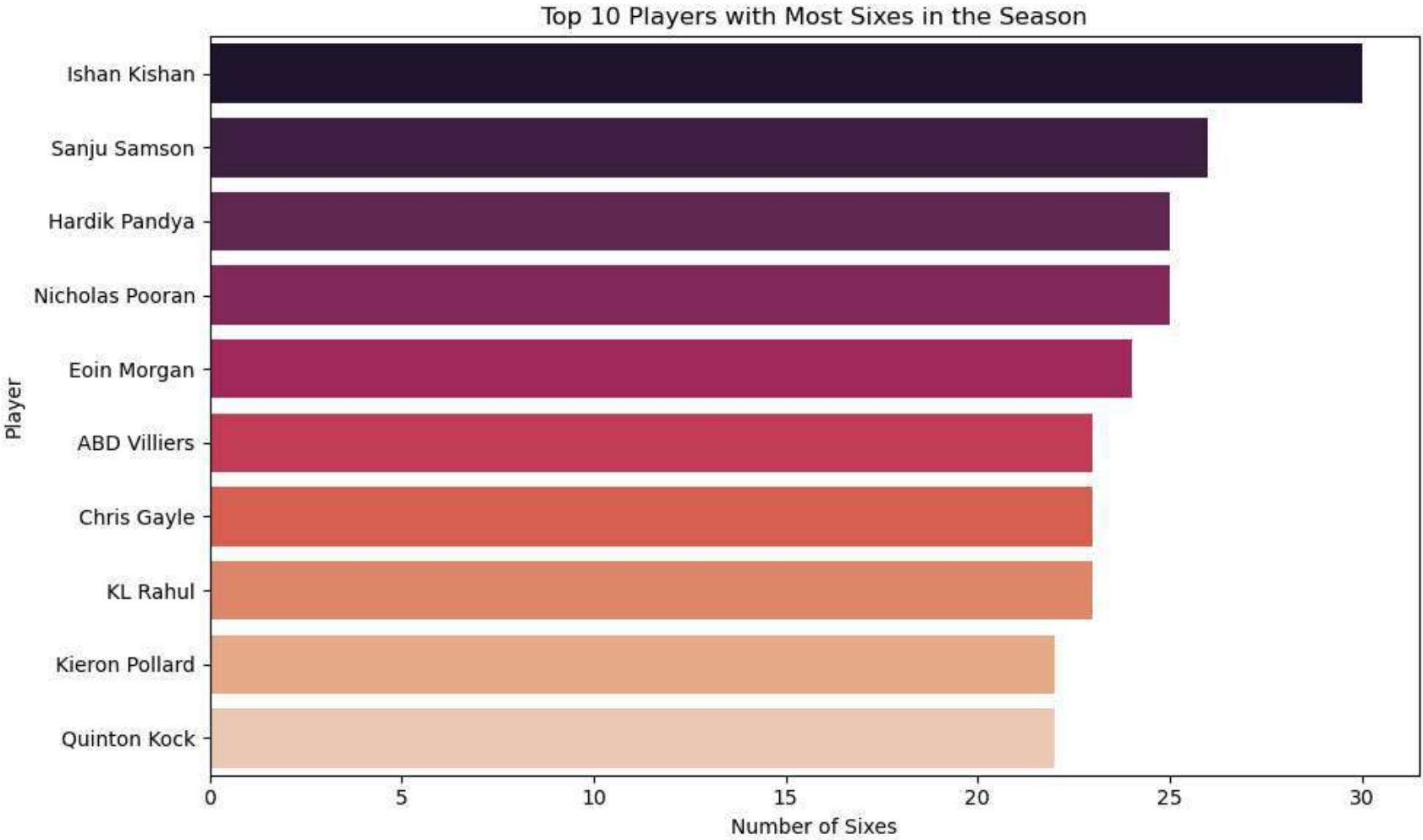
```
top_six_hitters = df_ip1.sort_values(by='6s', ascending=False).head(10)[['Player', '6s']]

plt.figure(figsize=(10, 6))
sns.barplot(data=top_six_hitters, x='6s', y='Player', palette='rocket')
plt.xlabel("Number of Sixes")
plt.ylabel("Player")
plt.title("Top 10 Players with Most Sixes in the Season")
plt.tight_layout()
plt.show()
```

C:\Users\pc\AppData\Local\Temp\ipykernel_14496\2164795323.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

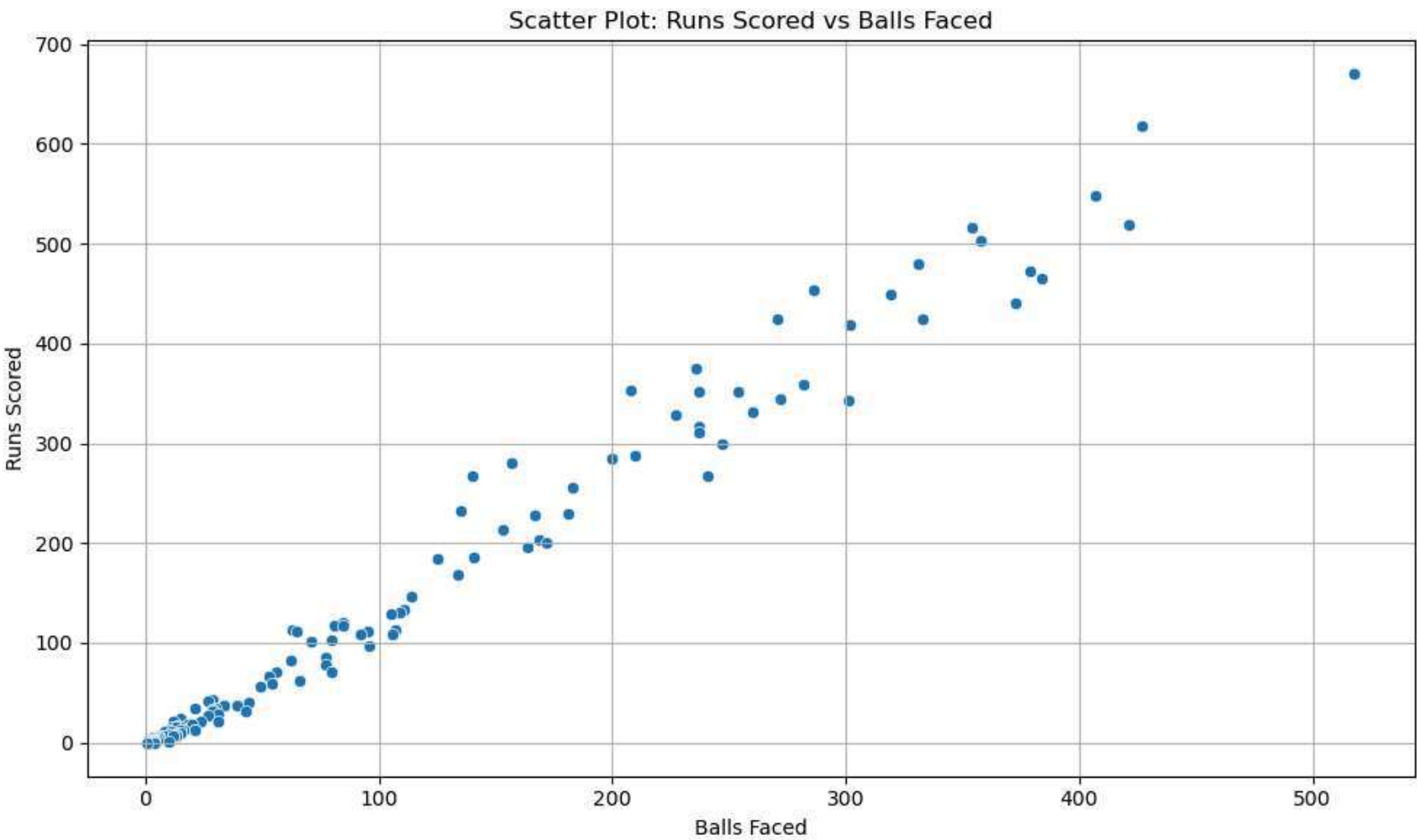
sns.barplot(data=top_six_hitters, x='6s', y='Player', palette='rocket')



Q20. Scatter plot of runs scored by a player v/s balls faced in a season. Then find the relationship between these 2 variables.

```
In [23]: plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_ip1, x='Balls faced', y='Runs')
plt.xlabel("Balls Faced")
plt.ylabel("Runs Scored")
plt.title("Scatter Plot: Runs Scored vs Balls Faced")
plt.grid(True)
plt.tight_layout()
plt.show()

runs_balls_correlation = df_ip1['Runs'].corr(df_ip1['Balls faced'])
runs_balls_correlation
```



Out[23]: 0.9899480233860845

```
In [ ]:
```